

MILESTONE 6: Free Exploration

- **Ackermann Steering:**

The bicycle model → a rear and a front, from 4 to 2

But in reality: the front inside wheel will take a smaller radius than the outside one during the turn: so the bicycle model becomes a little inaccurate

The Ackermann model accounts for this by trying to model the steering angle from the front differently from the back:

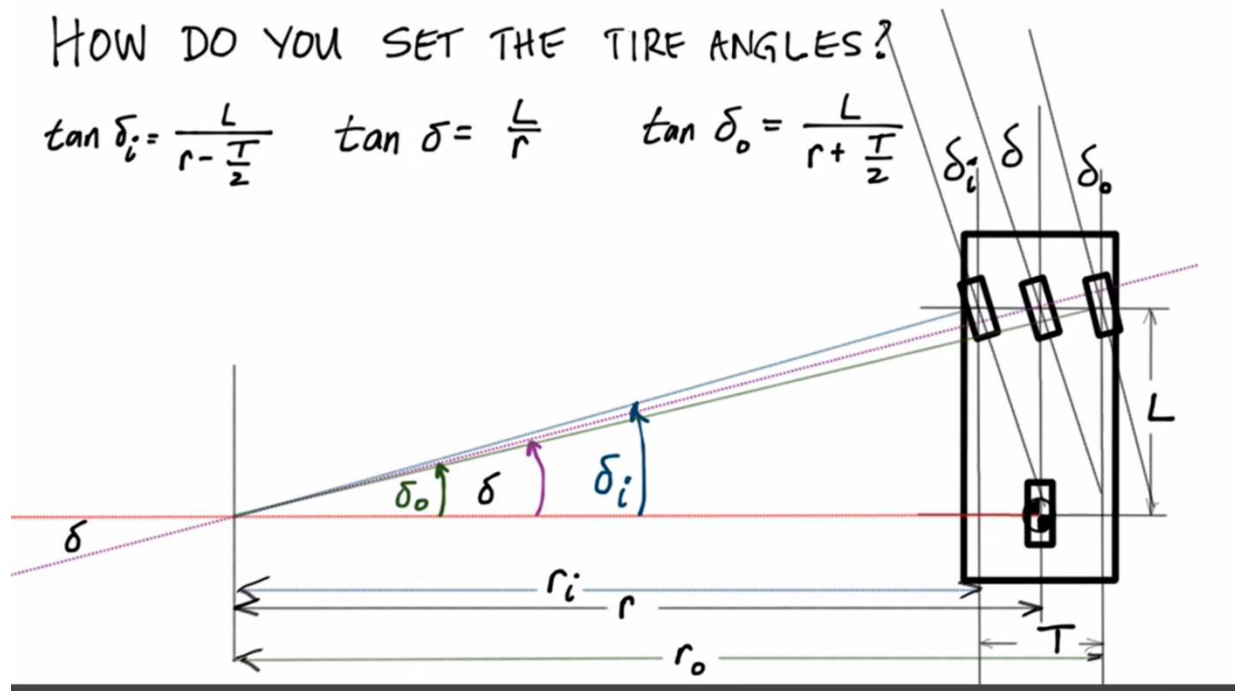


Figure 1: from https://youtu.be/i6uBwudwA5o?si=QQCpQwYgiHPn_KnU

ROS 2 Control Mobile Robot Kinematics Guide:

Omnidirectional robots

which can move instantaneously in any direction in the plane, and

Nonholonomic robots

which cannot move instantaneously in any direction in the plane.

In this case, we are focusing on **Nonholonomic robots**

In bicycle model: We assume that the wheels are rolling without slipping (the velocity of the contact point of the wheel with the ground is zero and the wheel's velocity points in the direction perpendicular to the wheel's axis). The **Instantaneous Center of Rotation (ICR)**, i.e. the center of the circle around

which the robot rotates, is located at the intersection of the lines that are perpendicular to the wheels' axes and pass through the contact points of the wheels with the ground.

$$v_{rear} = v_{front} \cos(\phi)$$

The front is faster because it moves along a longer path.

We have to distinguish between two cases: Encoders on the rear wheel or on the front wheel.

For the rear wheel case:

$$\begin{aligned}\dot{x} &= v_{rear} \cos(\theta) \\ \dot{y} &= v_{rear} \sin(\theta) \\ \dot{\theta} &= \frac{v_{rear}}{l} \tan(\phi)\end{aligned}$$

For the front wheel case:

$$\begin{aligned}\dot{x} &= v_{front} \cos(\theta) \cos(\phi) \\ \dot{y} &= v_{front} \sin(\theta) \cos(\phi) \\ \dot{\theta} &= \frac{v_{front}}{l} \sin(\phi)\end{aligned}$$

Forward kinematics answers: "If I know what the actuators are doing, how does the vehicle move?"

Inverse kinematics answers the opposite: "If I know how I want the vehicle to move, what should the actuators do?"

Ackermann steering:

The turning radius of the robot is

$$R_b = \frac{l}{\tan(\phi)}$$

Then the steering angles of the front wheels must satisfy these conditions to avoid skidding

$$\begin{aligned}\phi_{left} &= \arctan\left(\frac{l}{R_b - w_f/2}\right) = \arctan\left(\frac{2l \sin(\phi)}{2l \cos(\phi) - w_f \sin(\phi)}\right) \\ \phi_{right} &= \arctan\left(\frac{l}{R_b + w_f/2}\right) = \arctan\left(\frac{2l \sin(\phi)}{2l \cos(\phi) + w_f \sin(\phi)}\right)\end{aligned}$$

steering_controllers_library

This is a library designed for translating : needed car movement → wheel motor actions

It is mainly for (2 degrees of freedom: steering angle and velocity) and non-holonomic constraints (no sideways movement)

This library does not decide how to move the car, it takes the decisions and tells the motors how to do these actions (uses inverse kinematics)

It takes a TwistStamped (linear.x and angular.z)

Features:

- **Front and rear steering:** it works whether the steering wheels are at the front (a car) or the back (some forklifts).
- **Odometry publishing:** report the location of robot based on wheel movements → given as an Odometry message and as a TF message (a coordinate-frame transform).
- **Input command timeout:** if commands stop arriving, the controller resets the command to zero (reference_timeout)
- **Chain mode:** the controller can take its input from another controller instead of from a topic.

Type	Steering joints	Drive joints
Bicycle	1	1
Tricycle	1	2
Ackermann	2	2

A **joint** is a motor-controlled axis: a steering joint turns a wheel left and right, and a traction (drive) joint spins it.

Command interfaces: 1- steering joints → turn wheel left or right by x rate

2- drive joints → spin wheel by y rate

State interfaces: position_feedback: true → position false → velocity

Subscribers (only available not in chain mode)

Publishers

- <controller_name>/odometry [[nav_msgs/msg/Odometry](#)]

- <controller_name>/tf_odometry [[tf2_msgs/msg/TFMessage](#)]
- <controller_name>/controller_state [[control_msgs/msg/SteeringControllerStatus](#)]

CarlikeRobot

The front wheels → steering. Here, the front wheels **mimic the same angle**, so the example ignores that difference required in Ackermann model.

One steering input → virtual_front_wheel_joint

One driving → virtual_rear_wheel_joint

Instead of topics, it is uses an API to communicate with the robot with all data for all joints at once.

Live Example:

```

Code  Blame  27 lines (21 loc) · 759 Bytes  Raw  Copy  Download  Edit

1  controller_manager:
2  ros__parameters:
3      update_rate: 10 # Hz
4
5      joint_state_broadcaster:
6          type: joint_state_broadcaster/JointStateBroadcaster
7
8
9  bicycle_steering_controller:
10  ros__parameters:
11      type: bicycle_steering_controller/BicycleSteeringController
12
13      traction_joints_names: ['virtual_rear_wheel_joint']
14      steering_joints_names: ['virtual_front_wheel_joint']
15      wheelbase: 0.325
16      traction_wheel_radius: 0.05
17
18      reference_timeout: 2.0
19
20      open_loop: false
21      position_feedback: false
22      velocity_rolling_window_size: 10
23      base_frame_id: base_link
24      odom_frame_id: odom
25      enable_odom_tf: true
26      twist_covariance_diagonal: [0.0, 7.0, 14.0, 21.0, 28.0, 35.0]
27      pose_covariance_diagonal: [0.0, 7.0, 14.0, 21.0, 28.0, 35.0]

```

example_11/bringup/config/carlikebot_controllers.yaml

so these are the parameters discussed in the steering_controllers library.

example_11/hardware/carlikebot_system.cpp reads states and writes commands

Modern 3D Simulation Environments (Gazebo & MVSIM)

The bicycle model assumes that the tires never slip which is inaccurate to the real world, if we slip too fast in a certain corner the car could completely slip off the track.

Slip angle: angle between where the wheel points and where it travels. Never zero in real life because rubber behavior.

Friction Saturation: a tire can produce a certain amount of sideways grip. past it the car slips or understeers

Pacejka Magic Formula: (used in the paper in control task) predicts and models tire grip.

Gazebo: 3D simulator that models real life physics for robots

Controller – publish commands to module → Gazebo → sends sensor data to ROS2 → read by controller that decides next command

The bridge between Gazebo and controller → `gz_ros2_control`

So Gazebo is just like the `bicycle_model.py`

<https://github.com/alitekes1/ackermann-vehicle-gzsim-ros2> notes:

instead of bicycle → steering Ackermann

2D model → full 3D Gazebo model modelling dynamics around the car

It also uses sensors such as LiDAR, IMU to estimate where it is rather than the simulator providing this info like our project.

`saye_description` → this folder stores all the things related to the modelling, how it looks, its place etc for every single element

`saye_control` → contains controller node and a client subscribed to a service to get a map

`saye_msgs` → the map service itself

MPPI:

- Main idea:
 - Type of MPC that tries to minimize cost optimally
 - Core: sample sequences +noise → simulate them → find cost for each → make the plan go toward the lowest cost
- Algorithm:
 1. **Sample noise:** $\delta u_{k,t} \sim N(0, \Sigma)$ for K rollouts, T time steps
 2. **Roll out:** $x_{t+1} = f(x_t, u_t + \delta u_{k,t})$
 3. **Score:** $S_k = \text{terminal cost} + \Sigma \text{ stage costs}$
 4. **Weight:** $w_k = \text{softmax of the costs}$
 5. **Update:** $u_t \leftarrow u_t + \Sigma_k w_k \cdot \delta u_{k,t}$, then apply u_0 , shift the sequence forward, repeat
- Main formula for getting weight:

MPPI: Update nominal input as:

$$\mathbf{u} = \mathbf{u} + \frac{\sum_k w_k \delta \mathbf{u}_k}{\sum_k w_k}$$

Where weights are

$$w_k = e^{-\frac{1}{\lambda} S_k}$$

S_k : cost of trajectory k.

λ : constant parameter.



- **Dividing by the sum:** weights add to 1, so it's a proper weighted average
- Temperature (lambda): small → best path only matters → makes output jittery, Large → all matter equally so the controller does nothing
- The exponential transform (psi) : nonlinear → linear (Feynman-Kac: solutions of that linear equation are averages over random paths)
- Result: the optimal control is the weighted average of the paths, weighted by $\exp(-\text{cost}/\lambda)$
- It keeps re-weighting according to the luckiest path
- **Pros**
 - No gradients:** discontinuous costs and collision checks are fine
 - Parallel:** rollouts are independent, so it suits GPUs
 - Any dynamics model:** nonlinear, learned, or simulator

Simple to code

- 9. Cons

Needs many samples: too few gives jittery control

Model-dependent: garbage model, garbage plan

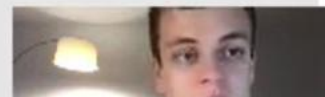
Local optimizer: it only explores near the nominal plan and can miss the “other side of the obstacle” route

Tuning: λ , Σ , T , and K all interact

-

```
Initialize control sequence  $\mathbf{u}_{0..N-1}$ 
while task not completed do:
    Generate random perturbations  $\delta \mathbf{u}$ 
    for Monte Carlo rollouts  $k=1..K$  do:
        start in current state  $\mathbf{x}_{k,0}=\mathbf{x}(t_0)$ 
        for MPC horizon steps  $n=0..N-1$  do:
            Input  $\mathbf{u}_{k,n} = \mathbf{u}_n + \delta \mathbf{u}_{k,n}$ 
            Next state  $\mathbf{x}_{k,n+1} = \text{model}(\mathbf{x}_{k,n}, \mathbf{u}_{k,n})$ 
            Rollout cost  $\mathbf{S}_k +=$  stage cost  $\mathbf{q}_{k,n}$ 
        end
    end
    for  $n=0..N-1$  do:
         $\mathbf{u}_n +=$  reward-weighted perturbations
    end
    Apply first input  $\mathbf{u}_0$ 
    Get system feedback

end
```



- NAV2 MPPI
 - nav2_mppi_controller → implements the MPPI algorithm you just studied.
 - It is local control so it needs global picture.
 - Global planner gives path → it outputs velocity commands

K (samples)	batch_size (default about 2000)
T (horizon steps)	time_steps (default about 56)
Time per step	model_dt (default about 0.05 s)
Dynamics $f(x,u)$	motion_model: can be Ackermann
Noise Σ	vx_std, vy_std, wz_std
λ (temperature)	temperature (lower = pickier)
Cost S_k	Sum of critics (penalties)
Warm start	Reuses the previous control sequence

Critics :

- **Obstacle/Cost critic:** penalizes trajectories that go through high-cost costmap cells or collide.
- **Path align / Path follow / Path angle critics:** keep the robot close to the global path and heading along it.
- **Goal / Goal angle critics:** pull the robot toward the goal position and orientation near the end.
- **Prefer forward critic:** discourages reversing.
- **Constraint critic:** respects velocity limits.
- **Twirling critic:** discourages spinning in place.